



**THE UNIVERSITY OF
JORDAN**



SADES: A Simplified Hybrid Block Cipher Combining S-AES and S-DES

Cryptography - (1911241)

Spring 2025/2026

Team Members:

Khaled Faisal Bdeir – 0248757

Zaid Abdulsalam Saleh - 0248736

Instructor: Dr. Oraib AbuAlganam

SADES: A Simplified Hybrid Block Cipher Combining S-AES and S-DES Encryption Methods

Khaled Faisal Bdeir*, Zaid Abdulsalam Saleh*

King Abdullah the II IT School, University of Jordan, Amman

Table of Contents

1. Abstract	2
2. Introduction	3
3. Methodology	3
o 3.1 Key Generation (S-AES Style)	3
o 3.2 Encryption Process (S-DES Style)	5
4. Security Analysis	6
o 4.1 Confusion and Diffusion	6
o 4.2 Block Cipher Design Principles	6
o 4.3 Comparison with Standard S-DES	7
5. Code.....	8
6. Conclusion	9
7. References.....	9

1. Abstract

This report introduces a simplified hybrid block cipher method Simplified Advanced Data Encryption Method (SADES), which combines the strength of both Simplified DES (SDES) and Simplified AES (SAES). SADES uses a modified key generation of SAES to produce two 8-bit keys along with the standard SDES encryption to encrypt an 8-bit plaintext. The report also goes over the methodology for encryption and how the new encryption method contributes to Confusion and Diffusion to produce a strong cipher method.

2. Introduction

Traditional SDES is educational for the purpose of understanding Feistel ciphers [1], but suffers from a small 10-bit key space, where the relation between the 10-bit key and the two 8-bit subkeys is based purely on linear operations such as bitwise shifting, permutations, or XORing with constants. Key generation in SDES has NO Substitution which is a simple but very weak (no confusion) [3]. By using the SAES key generation, the subkeys are generated using both permutation and substitution (using nibble boxes) which is a lot stronger than traditional SDES subkey generation.

SADES uses the strength of both SAES and SDES to improve on the traditional SDES. The new cipher method encrypts an 8-bit message using a 16 bit-key.

3. Methodology

3.1 Key Generation (S-AES)

SADES utilizes a 16-bit key to generate two 8-bit subkeys to be used in the following step (K1 and K2). This process does not only depend on simple bit-shifting, it also uses substitution to strengthen the subkeys and apply confusion through the usage of the nibble box.

Example Parameters

- **16-bit Master Key:** B3A7
- **Nibble Box (S-box):** Standard S-AES
- **Rcon (Round Constant for K2):** 80

Step 1: Initial Split

The 16-bit master key is split into two 8-bits W0 and W1.

- **(First 8 bits [W0]):** B3
- **(Last 8 bits [W1]):** A7

Step 2: Generate Round Key 1 (K1)

The first subkey is equal to W0 (K1=W0)

- (K1) = B3

Step 3: Generate Round Key 2 ()

The second key is derived by applying substitutions and permutations to W1 and combining the result with W0 ($W0 \oplus g(W1)$)

1. RotNib: Swap the two hex digits of W1

- $A7 \rightarrow 7A$

2. SubNib: Substitute each digit using the standard

S-AES nibble box

- $7 \rightarrow 5$
- $A \rightarrow 0$
- Result: 50

3. XOR the result with the Round Constant (80).

- $50 \oplus 80 = D0$

4. Final XOR (Result): Combine with the W0 to obtain K2

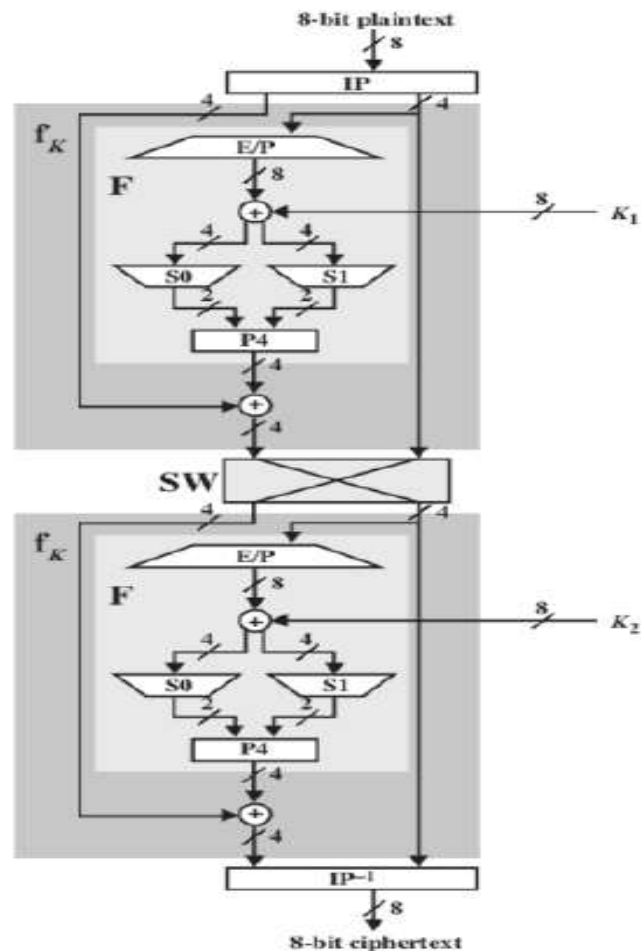
- $B3 \oplus D0 = 63$
- (K2) = 63

nibble	S-box(nibble)	nibble	S-box(nibble)
0000	1001	1000	0110
0001	0100	1001	0010
0010	1010	1010	0000
0011	1011	1011	0011
0100	1101	1100	1100
0101	0001	1101	1110
0110	1000	1110	1111
0111	0101	1111	0111

Fig 1. Standard S-AES nibble box [1]

3.2 Encryption Process (S-DES)

The encryption maintains the same standard S-DES encryption except that the K_1 and K_2 are the ones generated from the S-AES key generation step (3.1) to ensure that implementation is still simple



IP							
2	6	3	1	4	8	5	7

IP^{-1}							
4	1	3	5	7	2	8	6

E/P							
4	1	2	3	2	3	4	1

S0	00	01	10	11
00	1	0	3	2
01	3	2	1	0
10	0	2	1	3
11	3	1	3	2

S1	00	01	10	11
00	0	1	2	3
01	2	0	1	3
10	3	0	1	0
11	2	1	0	3

P4			
2	4	3	1

Fig 2. Standard S-DES Encryption process [1]

4. Security Analysis

4.1 Confusion and Diffusion

SADES contributes to both Confusion and Diffusion

- **Diffusion:** The statistical structure of the plaintext is dissipated into long-range statistics of the ciphertext, it is employed for increasing the redundancy of the plaintext by using permutation or transposition [1]. Like using **RotNib** in the key generation step.
- **Confusion:** Seeks to make the relationship between the statistics of the ciphertext and the value of the encryption key as complex as possible [1]. This is done through substitution like **SubNib** in key generation step.

4.2 Block Cipher Design Principles:

- **Strict Avalanche Criterion (SAC):** This principle states that any output bit of an S-box should change with probability $\frac{1}{2}$ when any input bit is inverted [1]. This is done by using the SubNib (nibble box) in S-AES subkey generation
- **Bit Independence Criterion (BIC):** This principle states that output bits j and k should change independently when any single input bit is inverted [1]. This is also done by using the SubNib (nibble box) in S-AES
- **Entropy:** measurement of the uncertainty, randomness or predictability of a ciphertext [2]. SADES provides higher entropy because it uses a bigger key which means more combinations.

4.3 Comparison with Standard S-DES

Feature	S-DES	SADES
Key Space	10-bits	16-bits
Key Combinations	$2^{10} = 1024$ combinations	$2^{16} = 65536$ combinations
Security Level	Educational	Strong
Resistance to Brute-force attacks	Minimal	Strong
Plaintext	8-bits	8-bits
Implementation	Simpler	More Complex
SAC (key)	Weak	Strong
BIC (key)	Poor	Independent bits
Entropy	7.6065	7.9089

- We notice that SADES provides a bigger key, more security, more resistance to brute-force attacks.
- SADES also makes sure that the block cipher design principles are applied, which are SAC, BIC and Entropy.
- The implementation is a little more complex now but still very simple and not as heavy as S-AES.

5. Subkey Generation Code Implementation

```
# KEY GENERATION
def rot_nib(byte):
    high = (byte >> 4) & 0xF
    low = byte & 0xF
    return (low << 4) | high

def sub_nib(byte):
    high = sbox[(byte >> 4) & 0xF]
    low = sbox[byte & 0xF]
    return (high << 4) | low

def generate_keys(master_key_hex):
    master_key = int(master_key_hex, 16)

    W0 = (master_key >> 8) & 0xFF #first byte
    W1 = master_key & 0xFF #second byte

    K1 = W0

    after_rot = rot_nib(W1)
    after_sub = sub_nib(after_rot)
    g_W1 = after_sub ^ RCON
    K2 = W0 ^ g_W1
```

Fig 3. SADES subkey generation code implementation in Python

```
***** Key Generation *****
Master key = B3A7
W0 = B3, W1 = A7
RotNib(W1) = 7A
SubNib(7A) = 50
XOR with RCON: 50 XOR 80 = D0
K2 = W0 XOR g(W1) = B3 XOR D0 = 63
K1 = B3, K2 = 63
```

Fig 4. The output of the code given in Fig 3.

6. Conclusion

The SADES cipher upgrades the old S-DES significantly by replacing the 10-bit key with a 16-bit key and applying a modified S-AES subkey generation to employ both Diffusion and Confusion, apply Strict Avalanche Criterion (SAC) and Bit Independence Criterion (BIC), and improve entropy (uncertainty/randomness of the cipher method).

7. References

- [1] Cryptography course slides – Chapter Three: Modern Encryption Techniques
- [2] Stallings, W. (2023). Cryptography and network security: Principles and practice (8th ed.). Pearson.
- [3] Understanding Cryptography: A Textbook for Students and Practitioners
Paar, C., & Pelzl, J. (2010). *Understanding cryptography: A textbook for students and practitioners*. Springer.